LANTERN full project log

Sensor Trust for Lunar South Pole Landing. Complete chronological record of what was done, decided, found, got wrong and fixed. Written 19 Oct 2026. Short versions: `PROGRESS.md` (status), `Progress_Report.pdf` (results), `Paper_Draft.pdf`, `PREREG.md`.

0. Starting point (before 18 Oct)

- Five plan documents in `deliverables/`: literature review, research gap, objective and problem definition, proposed methodology, paper summaries.

They define the problem (HAKUTO-R 2023 rejected valid altimeter readings at a crater rim; IM-2 2025 trusted a camera in shadow), the four-step trust rule, objectives O1-O7, hypotheses H1-H4, metrics and the fair-testing rules.

- NASA PGDA Product 78 Site04 elevation and slope GeoTIFFs in `data/` (5 m/pixel, 3200 x 3200, -2848 m to +1805 m). Rendered by `tools/render_site.py` (`out/site04/`).
- A Unity 6 project `LANTERN/` already existed: Site04 scene built from the DEM, a rigid-body lander (`LanderController`), HUD, follow camera,

and the LUMEN live-scan module (lidar raycasts, height patch, patch matcher, position correction).

1. Unity connection and smoke test (18 Oct)

- Checked the Unity MCP connection: project `D:/newprj/LANTERN`, scene `Site04.unity`, scripts as above. Connected.
- **Smoke test failed first**: entering Play mode left the game loop frozen at frame 2. Cause: the Unity editor window reported itself inactive

(`isApplicationActive` false). Fix: `Application.runInBackground = true`. Later made automatic in `DescentAutopilot.Start`.

- After the fix the lander free-falls with throttle 0 (at t = 16.6 s: 276 m above ground, -26.9 m/s, fuel 800 kg). Console clean.

2. Landing site selection (18 Oct)

- The supplied slope file `Site04_final_adj_5mpp_slp.tif` is **truncated** (30.5 MB instead of 41 MB); the missing part loads as 0 degrees, which fakes flat areas.

`tools/render_site.py` already handles this by deriving slope from the DEM; the same was done here.

- Tile centre: 33 degrees, 35 degrees maximum over 45 m. Only 0.4% of the tile has a 45 m maximum slope under 5 degrees.
- Chosen site: Unity (x 6198, z 8512), 1.88 km from the centre, elevation 1729.5 m, mean slope 1.1 deg, 45 m-window maximum 2.8 deg.

The Unity terrain height there equals the DEM (1729.5 m). Lander start moved there (scene saved, `Site04SceneBuilder.cs` updated).

3. HUD, shadows and autopilot (18 Oct)

- HUD text was white on sunlit near-white terrain and too small. Added a dark backing box, then scaled the font to 3% of screen height with a box sized to the text.

Verified on a 1920 x 1080 game-view screenshot (`out/hud_check.png`).

- Low-Sun cast shadows verified with camera captures (shadow edges visible; shadowed side black because ambient light is 0).
- `DescentAutopilot.cs`: free-fall coast, speed-profile braking, upright hold. First run from 500 m touched down at **2.07 m/s** (over the 2 m/s target);

retuned (final-descent rate 0.5 m/s, derating 0.5, velocity gain 4): **0.50 m/s**, 0 m/s horizontal, 0.0 deg tilt.

4. 102.5 km start and time warp (18 Oct)

- User asked for a start 100-105 km above the surface. Lander moved to 102.5 km above the flat spot, dropped from rest.

Autopilot gained time warp (8x high up, 1x below 4 km, physics step unchanged) and checkpoint logging.

- Full run: free fall about 190 s to 312 m/s at 70 km, then about 0.69 throttle to touchdown. Touchdown t = 583-587 s, 0.50 m/s vertical,

0 m/s horizontal, 0.0 deg tilt, **251 kg fuel left** (549 kg used), exactly on target.

- Caveat: a real descent from 100 km altitude starts with about 1.7 km/s horizontal speed. The autopilot does not handle that (vertical drop only).

5. LUMEN diagnosis and fix (18 Oct)

- Found dead zones: invalid at 1252 m, at 25 m and at touchdown. Cause (from code): lidar beam spacing grows with altitude (about 14 m at 1250 m) while the

patch cell was fixed at 8 m, so cells never got 2 points; and below about 80 m the scan footprint is too small (the map itself is 7.8 m/pixel).

- Fix: patch cell scales with beam spacing (`adaptiveCell`, `cellPerBeamSpacing` 1.6). Result: first fix at 2463 m;

error 47 m at 2205 m (41 m cell), 14.5 m at 921 m, 4 m at 155 m (8 m cell). Over 108 logged fixes: median 8.9 m, max 48.7 m, injected error (+60, -40) m,

shift reported (-64, 32) m, confidence 1.00. No fix below about 80 m (physical limit, left as is).

6. Data logging (18 Oct)

- `TelemetryLogger.cs`: one CSV per Play session in `out/runs/` plus `runs_summary.csv`, flushed per row.
- Bug found: the first run's summary recorded start altitude 0 (the logger read the autopilot's altitude before the autopilot had computed it). Fixed afterwards

(logger now computes altitude itself); not re-verified in a later Unity run.

7. First documents and git (18 Oct)

- Project description and progress PDFs built with reportlab (`tools/make_report_pdfs.py`); pages rendered and checked.
- Git initialised in `D:/newprj`; `.gitignore` excludes Unity `Library/Temp/Logs/UserSettings`, the large GeoTIFFs and caches. Private repo

`AnakinSkywalker0/lantern-sensor-trust` created and pushed (user chose private, exclude large files).

8. Reading the plan and scoping the full study (18-19 Oct)

- The user said "complete everything". Reading `4_Proposed_Methodology.pdf`: the plan requires a **fast Python test bench for the 500-run Monte Carlo**

(rule 7), Unity only replays selected landings; scope is powered descent from about 15 km; tune on Site01 plus procedural terrain, test on Site04.

- Decisions and disclosures made at the time: the Python bench starts at 15 km (the Unity demo stays at 102.5 km); Site01 missing so tuning uses procedural terrain only;

HAKUTO-R altitude record unavailable; guidance and sensor settings were chosen while looking at Site04 runs (disclosed in every report since).

9. Python test bench (18-19 Oct)

- `sim/world.py`: Site04 and procedural crater worlds (10 m), truth surface with unmapped craters and roughness, onboard map banks (3 m height noise, registration shift),

coarse reference surface (4 km Gaussian), shadow rasters at 20 m (cached), region statistics (min/max/std, lit fraction at several radii), track selection

(about 10 km from high ground across the rim to a flat floor: start 15 km above the rim, target on the crater floor).

Site04 coarse reference reads -1482 m at the target against a true -2755 m (this offset is the HAKUTO-R mechanism).

- `sim/engine.py`: batched simulator (500 runs in lock-step), truth physics (1.62 m/s2, 2000 kg, 6000 N, Isp 300 s), IMU with bias, altimeter with fault bursts,

delay and dropout, camera and live-scan fixes, a 9-state EKF, guidance on the estimate (speed profile with a 15 m constant-rate final), and seven judges:

fixed, PITFAL-style, map tolerance, wait-and-see, second opinion, ML classifier, trust (four-step). Ablations of the trust rule.

- Smoke tests found and fixed: guidance had no constant final descent (touchdown 2-3 m/s); with the Sun from the south-east the track never entered shadow,

so the default Sun was set to the west (az 270) for the IM-2 scenario.

10. Round 1: pipeline, results and the mistake in it (19 Oct)

- `sim/pipeline.py`: stages ml, tune, main, sweeps, replay (parallel). First launch with 20 workers ran out of memory (15.7 GB machine, 5-6 GB free);

cut to 6 workers and one cached world per worker. At the user's request ("do not train anything right now") all Python processes were stopped; no model existed;

training was resumed only after the user said "execute them all".

- ML classifier (round 1): 77.7% held-out accuracy, AUC 0.85. Tuning on procedural worlds chose probation 8 samples, 4 sigma.
- Round-1 results (Site04, 500 runs): fixed 0.4%, PITFAL-style 49%, trust layer 89% (cliff); the "controlled" check giving every judge the same shadow-aware camera showed

the map-tolerance gate at 93% against the four-step rule's 89%; with a mildly damaged camera PITFAL-style reached 94-100%. Everything in `out/mc_round1/`, tag `round1`.

- **The mistake**: round 1's camera-in-shadow damage curve was my assumption, and the comparison gave the trust layer a shadow-aware camera while the other judges fed every

camera fix to the filter. Round 2 corrected this (section 14).

- Replay export: found a recording bug (the filter kept integrating free-fall after touchdown, so the estimate drifted km below ground in the replay); fixed to stop recording at contact.
- Unity replay scene: `ReplayPlayer.cs` and `ReplaySceneBuilder.cs` (builds `Site04Replay.unity`). Verified by play and screenshots: the fixed-gate lander hovers 1372 m up believing it is

62 m below the surface while the other two land.

- Documents and push: progress report with conclusions (6 pages), `PROGRESS.md`, README; pushed. The user asked for conclusions in plain language; they were given.

11. Round 2: why and what changed (19 Oct)

- User: "do it all". Next-step list: (1) justify the camera model, (2) redesign probation and add timestamps, (3) fix the no-map fallback, (4) get more data, (5) write the paper.

11.1 Site01 data (user approved the download)

- Asked before downloading (filename, source, size). Approved: `Site01_final_adj_5mpp_surf.tif` and `..._toterr.tif` (40,980,810 bytes each) from

pgda.gsfc.nasa.gov/data/LOLA_5mpp/Site01/ into `data/` (not in git). Checked: 3200 x 3200, no gaps; relief -523 to 1959 m; NASA total error 0.06-5.6 m.

The error map sets the map-noise level for Site01.

11.2 Camera survey in Unity

- `sim/cam_survey_plan.py` picks 300 positions on Site04 stratified by lit fraction (60 per bin). `CameraSurvey.cs` renders paired nadir images: live (true Sun,

displaced by an unknown offset) and reference (onboard prediction, Sun 0.05-0.25 deg higher, +-1 deg azimuth).

- First pass showed dark scenes with mean 21/255, from the skybox leaking into shadows. Fixed by switching reflections and ambient intensity off during the survey,

then re-rendered; dark scenes became flat (texture std 0.8).

- `sim/cam_survey_match.py`: ZNCC landmark matcher with sensor noise (read noise 2 DN, shot noise). Two reference variants: direct-light-only (primary) and scattered-light (optimistic).

Primary result: median error 401 m and 88% of fixes off by more than 100 m when the footprint is under 5% lit; 7.6 m (20% off by >100 m) at 5-25%; 2.4 / 1.3 / 0.8 m above 25%.

The matcher confidence (correlation peak) was a perfect failure detector: 0 of 226 fixes with peak >= 0.5 were off by more than 100 m.

- **Second correction**: the first round-2 smoke test showed PITFAL-style at 0%, because it fed every fix to the filter. That is a strawman (a real matcher reports confidence).

Camera policies were added: naive, confidence gate, Sun-geometry prediction, both. Every judge now gets the confidence gate by default.

11.3 Trust layer v2

- Noise level learned online from accepted readings (fixes noisy-but-valid readings); time-stamped readings compensated for delay (assumes the message carries its acquisition time);

no-map fallback that tracks ground height under the lander.

- The no-map fallback took three attempts, debugged on a procedural world only: (a) ground height reset on every accepted reading left vertical speed unobservable

(AGL estimate good, vertical speed off by 1.4 m/s, touchdown 2.8 m/s); (b) a terrain-relative altitude filter for the whole descent flew into the crater wall; (c) final approach

only on the terrain-relative filter below 200 m: 98% on the tuning world.

- ML classifier retrained with the new camera: 89.5% held-out accuracy, AUC 0.97 (procedural worlds only).

11.4 Pre-registration and freeze

- `PREREG.md` written before the Site04 run: changes since round 1, frozen settings, primary hypotheses P1-P6 with decision rules, procedure.

Disclosure in it: a 100-run Site04 smoke test of the new engine was inspected before writing it (trust v2 looked ahead; no-map still 0% at that time).

- Tuned on procedural worlds + Site01 only (grid K 8/10/12, sigma 2.5/3/4): chosen K = 10, sigma = 4 (87% mean success there). Committed, tagged `prereg-round2`, pushed.

11.5 The Site04 evaluation (run once)

- 500 runs per cell, 9 judges + 2 ablations, 5 scenarios, confidence-gated camera for everyone; camera policy comparison; sweeps; replay.
- Pre-registered outcome: P1 met (+14.8 / +11.8 / +24.0 points over PITFAL-style), P2 mixed, P3 met in cliff cases and failed in shadow + noisy (52% vs 33% fooled),

P4 supported (Sun-geometry prediction 5-11 points worse than the confidence gate in three of four cases), P5 met, P6 met only via time stamps.

- Exploratory (added after the main run, labelled as such): giving the other judges the same time stamps: tolerance gate 97.6%, wait-and-see 79%, trust v1 79.6%,

PITFAL-style 74.2%; trust v2 without time stamps 0%. Only change to the code after the freeze tag: a `timestamps` option in `sim/engine.py`; default behaviour identical.

12. Headline numbers

Round 1 (kept for comparison; tag `round1`; unfair camera baseline), landing success %, Site04, 500 runs

Scenario	Fixed	PITFAL	Trust v1
Cliff	0.4	49.0	89.2
Cliff + faults	0.0	45.6	84.6

Scenario	Fixed	PITFAL	Trust v1
Shadow + noisy	0.0	27.0	55.0
Delayed	0.0	1.2	0.2
Dropout	0.0	7.8	44.4

Round 2 (pre-registered; confidence-gated camera for every judge), landing success %, Site04, 500 runs

Scenario	Fixed	PITFAL	Map tol.	Wait	2nd op.	ML	Trust v1	Trust v2	v2 + scan
Cliff	0.2	82.0	97.2	79.6	56.6	75.8	94.6	96.8	100.0
Cliff + faults	0.0	85.6	97.6	80.2	8.6	76.0	94.8	97.4	99.8
Shadow + noisy	0.0	64.0	61.6	28.6	15.6	23.8	53.4	88.0	59.2
Delayed (v2 time-stamped)	0.0	0.0	0.0	0.2	0.0	0.4	0.0	83.0	95.8
Dropout	0.0	34.8	41.6	29.0	26.2	24.2	31.4	37.0	61.4

Round 2 sweeps and robustness (300 runs; sun sweep 200)

Test	PITFAL	Trust v2
Position error 5 / 50 / 200 / 400 m (cliff)	81 / 83 / 76 / 61	97 / 98 / 95 / 96
Divert	74.3	91.3
No map	2.7	90.7
Map shifted 2 km	35.0	1.3
Site01 vs Site04, cliff	100 vs 81.7	100 vs 98.0
Sun from the east (well lit), shadow + noisy	99.5	39
Sun from south / west, el 1.5-3 deg	62-70	81-90

13. Conclusions

1. The HAKUTO-R failure is reproducible: a fixed gate with a coarse reference fails 99.8% of landings.
2. The map in the filter does most of the work (82%), and a plain map-tolerance gate reaches 97%.
3. The four-step rule matches but does not beat that gate in cliff cases; probation adds nothing there.
4. Trust v2 wins with a noisy altimeter in shadow, with no map and on a divert; it fails with a noisy altimeter in good light and with a shifted map, and is fooled by more faults in the noisy case.
5. Camera fixes must be gated by matcher confidence (ungated: 0.4-3%); Sun-geometry prediction adds nothing over it.
6. Delayed readings are solved by time stamps, not by the trust logic.
7. Round 1's shadow-aware-camera headline did not survive a fair baseline.
8. Publishable as a controlled-comparison workshop paper with honest negative findings; not a breakthrough. Simulation only.

14. Errors, corrections and disclosures (all of them)

What	When found	What was done
Play mode frozen (editor inactive)	18 Oct	`runInBackground`, later automatic
Truncated slope file read as flat	18 Oct	slope derived from the DEM
HUD unreadable on bright terrain	18 Oct	dark box, font scaled to screen
Autopilot touched down at 2.07 m/s	18 Oct	retuned to 0.50 m/s
LUMEN blind above ~500 m	18 Oct	patch cell scales with altitude

What	When found	What was done
Logger recorded start altitude 0	18 Oct	fixed; not re-verified in Unity
20-worker pool ran out of memory	19 Oct	6 workers, one world per worker
Replay recorder kept integrating after touchdown	19 Oct	stops at contact
Round-1 camera model assumed and unfair baseline	19 Oct (round 2)	replaced by measured model and confidence-gated camera; round 1 kept and labelled
Skybox light leaked into survey shadows	19 Oct	reflections and ambient off, re-rendered
No-map fallback unobservable vertical speed	19 Oct	terrain-relative altimeter filter on final approach
Design looked at Site04 smoke runs before pre-registration	19 Oct	disclosed in `PREREG.md` and every report
Time-stamp control added after the main run	19 Oct	labelled exploratory; only a default-off code option changed after the freeze tag
`sim/engine.py` differs from the `prereg-round2` tag	19 Oct	disclosed; default behaviour identical
PDF bullets rendered as mojibake	19 Oct	fixed in the generator; all PDFs re-checked
Two statements in the report slightly off (camera prediction "within noise"; scan "up to 20 points")	19 Oct	corrected before commit

15. Open items

- Repair trust v2's two weaknesses: scale the physical check with the noise level; detect a wrong map.
- Terrain-relative no-map baseline for a fair comparison.
- Repeat the camera survey on Site01 and with a second matcher; add a scattered-light prediction case.
- Real HAKUTO-R altitude record (not available). Site01 only used for tuning, so Site04 is the only unseen real site.
- Optional: PGDA Product 104 as a second high-detail site; an orbital-speed start for the Unity demo; an outside read-through before submission.

16. Where everything is

Path	What
`LANTERN/Assets/Scripts/`	`LanderController`, `DescentAutopilot`, `LanderHud`, `TelemetryLogger`, `FollowCamera`, `ReplayPlayer`, `Lumen/`
`LANTERN/Assets/Editor/`	`Site04SceneBuilder`, `ReplaySceneBuilder`, `CameraSurvey`
`LANTERN/Assets/Scenes/`	`Site04.unity`, `Site04Replay.unity`
`sim/`	`world.py`, `engine.py`, `pipeline.py`, `plots.py`, `cam_survey_plan.py`, `cam_survey_match.py`, `smoke.py`, `quicktest.py`, `dbg_nomap.py`
`tools/`	`render_site.py`, `make_heightmap.py`, `make_report_pdfs.py`
`docs/`	`PREREG.md`, `PROGRESS.md`, `FULL_LOG.md` (+ PDF), `Progress_Report.pdf`, `Paper_Draft.pdf`, `Project_Description.pdf`
`out/mc/`	round-2 results (main_results.csv/.md, camera_policies, sweeps, exploratory_timestamps, tuning, ML report, per-run CSVs)
`out/mc_round1/`, `out/plots_round1/`	round-1 results (tag `round1`)
`out/camera_survey/`	survey positions, matcher results, empirical camera model (images not in git)

Path	What
`out/runs/`	Unity telemetry CSVs and run summaries
`out/plots/`, `out/replay/`	figures and replay data; `out/replay_round2.png` Unity replay capture
`data/`	NASA GeoTIFFs (not in git)

Git tags: `round1` (6bbda5c), `prereg-round2` (ca61ea6). Reproduce round 2: `python sim/pipeline.py ml tune` then `main policies sweeps explore replay`, `python sim/plots.py`, `python tools/make_report_pdfs.py`.